
MP8: Postprocessing with Multiple Camera Passes

Due Time: Refer to the course web-site

Objectives

Verify that we understand and are able to work with:

- Unity specific skills:
 - There should be no new skills challenged or required
 - You do need to be confident in Material sharing and instancing
- Shader and relevant general concepts:
 - Work with multiple camera passes, with distinct camera settings and requirements
 - Implement multi-pass algorithms where scene information are computed and forwarded for later passes
 - Simulate shadow and reflection

Approach

Based on our MP7 solution, we will implement the basic shadow depth map, or shadow map, and reflection algorithms learned in class.

Shadow Map. We will take the prototype in the class example and integrate the algorithm into our Phong illumination model. We must support:

- Depth and Normal Bias
- Shadow Color
- Blending of shadow color with light illumination
- Percentage Closer Filtering, with 9x9 kernel.

Additionally (this is something I did not do), on the SceneControl, you must allow users the choice of map resolution: 128x128, 512x512, 1024x1024, and 2048x2048. I want you to observe the quality of shadow based on these different resolutions. It is acceptable for the user to select the resolution `_before_` running the system. If you choose to support dynamically changing the shadow map resolution, [make sure remember to release the resource](#), otherwise, your machine may not be happy with you.

Reflection. Integrate the effect into the same scene. The reflection and shadow should work in the same scene. You must support the option of switching off the mirroring effect. More importantly, the material (or shader) on the mirror must be consistent and must be Phong. That is, if the mirror effect is switched off, you will see a simple Phong illuminated plane, you can change its color, texture, etc. When the mirroring effect is switched on, the only addition will be reflecting the scene (with material settings preserved). Once again, your user can adjust appearance of the mirror (e.g., changing its diffuse color), and the changes must be preserved when the mirroring effect is switched off. Remember, the mirror object (the plane) will be rendered by three shaders:

- The default shader (Mirror.Shader): simple writing into stencil and ensuring relevant pixels have 0 in the alpha channel.
- The MirrorDraw shader: this is the shader that will be used when reflect camera is drawing. This shader will compute the mirror's color with Phong, and make sure to not write into the Z-buffer.
- The Non-Mirror shader: this is the shader that will be used when mirror effect is switched off. This will be a usual Phong shader.

You need to support the consistency of the output from the MirrorDraw and Non-Mirror shaders (same values).

Postprocessing Support. To illustrate (to ourselves) the generality of and issues with these two algorithms you will integrate these two effects with the GroundFog and Torch shaders from MP7. We can see that these effects simply works, and yet, the we will observe that the appearance in the mirror is inconsistent. For example, unless you do something special (try it if you have time) you will not see GroundFog in the mirror! And, it is possible to adjust the Torch parameters such that blurriness of the same object is different in the mirror (remember, blurring is done as a post process after the image is computed).

This assignment is significantly more work and trickier than the previous ones. If nothing else, we have much more assets to work with, and much more complicated code base for integration. As such, this assignment will weigh more, 5% more, than the other assignments.

Hints

- Ground fog looks bad (and wrong) as the ground plane size is not infinite. I did not spend time resolving the very large plane problem: large objects can be challenging for Shadow Map because of bad perspective space approximation for objects with size that approach the size of the frustum.
 - As ground plan size grows, the self-shadowing problem can become challenging to manage (same problem exists with Unity, or any of the shadow map implementation). One solution here is prevent the ground plan from participating in the initial depth-map camera rendering. I did not spend time working on this.
- Note the limitations of our depth map that the light direction **does not** affect the shadow.
 - Our depth map is like that from a spot light: rotating the depth map camera is like rotating the cone of a table-lamp without moving the lightbulb position.
 - Depth map is most accurate when the depth map camera views the shadow-receiving plan perpendicularly. For this assignment, have the camera looking straightdown.
- Please Implement the shadow map on Light-0, yes, you only need to implement one light with shadow map.
- Please **remove** all projection shadow support, or, make sure it can be clearly differentiate between the two types of shadow!

Credit Distribution

	Credits
1 Unity Scene Setting	18
At least 6 objects, with ground plane at $y = 0$	2
All objects are illuminated by our Phong model	
Verticle mirror	
At least 2 lights (one must be directional): on/off	4
Initial scene setting is reasonable	5
Proper lighting, no over-saturation, decent mirror view	
Reasonable controls (not difficult to find)	3
E.g., Controls on SceneControl	
Support MP7 post-processing: Fog + Depth	4
NO worries about consistent reflection	
Shadow	
2 Map	35
Basic Shadow Map working on Light-0	6
SceneControl: set resolution before run: 128, 512, 1024, 2048	8
Shadow color setting (in SceneControl)	4
Proper blending with shadow color	4
Percentage Closer Filtering (9x9) with on/off	5
Depth + Normal Bias Support	4
On/Off control (on Scene Control)	4

3	Reflection	34	
	Reflection in the vertical mirror		8
	Phong Shader Illuminated (with Texture, Diffuse, etc)		6
	Adjust Phong parameters for mirror and non-mirror		6
	Mirror On/Off (on Scene Control)		6
	Appearance consistent between on/off		8
	UI and		
4	Control	8	
	Post Process control integrated with SceneControl		8
	Toggle effects on/off, control sequence		
6	Submission		
	Proper folders/unity version		5
	Total		100

This assignment counts 15% towards your final grade for this class.